

EXPERIMENT NO 5

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

// Function to print the board

void printBoard(const vector<vector<int>>& board, int N) {

    for (int i = 0; i < N; i++) {

        for (int j = 0; j < N; j++)

            cout << board[i][j] << " ";

        cout << endl;

    }

}

// Function to check if placing a queen at (row, col) is safe

bool isSafe(const vector<vector<int>>& board, int row, int col, int N) {

    // Check same column

    for (int i = 0; i < row; i++)

        if (board[i][col] == 1) return false;

    // Check upper-left diagonal

    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)

        if (board[i][j] == 1) return false;

    // Check upper-right diagonal

    for (int i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++)

        if (board[i][j] == 1) return false;

    return true;

}

// Backtracking function to place queens

bool solveNQueens(vector<vector<int>>& board, int row, int N) {

    if (row >= N) return true; // All queens placed
```

```

// Skip row if queen already placed
if (find(board[row].begin(), board[row].end(), 1) != board[row].end())
    return solveNQueens(board, row + 1, N);

for (int col = 0; col < N; col++) {
    if (board[row][col] == 0 && isSafe(board, row, col, N)) {
        board[row][col] = 1; // Place queen
        if (solveNQueens(board, row + 1, N))
            return true;
        board[row][col] = 0; // Backtrack
    }
}
return false; // No valid position
}

int main() {
    int N;
    cout << "Enter size of board (N): ";
    cin >> N;
    vector<vector<int>> board(N, vector<int>(N, 0));
    int firstRow, firstCol;
    cout << "Enter position of first queen (row and column, 0-based index): ";
    cin >> firstRow >> firstCol;
    if (firstRow >= N || firstCol >= N || firstRow < 0 || firstCol < 0) {
        cout << "Invalid position!" << endl;
        return 0;
    }
    board[firstRow][firstCol] = 1; // Place first queen
    // Solve remaining queens starting from row 0
    if (solveNQueens(board, 0, N)) {
        cout << "\nN-Queens solution:\n";
    }
}

```

```
    printBoard(board, N);
} else {
    cout << "No solution exists with the first queen at the given position." << endl;
}
return 0;
}
```

OUTPUT

icoer@icoer-Vostro-3470:~\$ g++ n-queen.cpp

icoer@icoer-Vostro-3470:~\$./a.out

Enter size of board (N): 4

Enter position of first queen (row and column, 0-based index): 0

1

N-Queens solution:

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0